

PROJETO download_wfs

DAG: download_imoveis_sicar — download de dados WFS do SICAR (CAR)

Módulo	Valor
Tipo	DAG Airflow (PythonVirtualenvOperator + ShortCircuitOperator)
Agendamento	@daily (com re-disparo em loop via trigger_self)
Fonte de dados	https://geoserver.car.gov.br/geoserver/sicar/wfs
Tags	download, wfs
Banco de saída	conexão Airflow DOWNLOAD_WFS_DB (PostgreSQL)

1. Fluxo das tasks

O DAG possui 5 tarefas executadas em sequência. A última (**trigger_self**) re-dispara o próprio DAG enquanto a task 4 indicar que ainda existem UFs a processar — formando um ciclo de processamento diário.



Task	Operador	Arquivo	Papel
1_check_config	PythonVirtualenv	check_wfs_download_config.py	Valida a configuração e tabelas obrigatórias
2_wfs_download	PythonVirtualenv	task_2_wfs_download.py	Consulta WFS por UF/ano e baixa shapefiles (ZIP)
3_insert_shape_file	PythonVirtualenv	task_3_insert_shapefile.py	Descompacta, limpa e insere/atualiza geometrias no
4_check_if_trigger	ShortCircuit	task_4_check_if_trigger.py	Verifica se ainda há UFs pendentes
trigger_self	TriggerDagRun	dag.py	Re-dispara o DAG para a próxima rodada

2. Estrutura de diretórios

```
download_wfs/ ■■■■ __init__.py ■■■■ src/ ■ ■■■■ dag.py # definição do DAG e encadeamento
das tasks ■ ■■■■ download_imoveis_sicar/ ■ ■ ■■■■ check_wfs_download_config.py # task 1
■ ■ ■■■■ task_2_wfs_download.py # task 2 ■ ■ ■■■■ task_3_insert_shapefile.py # task 3 ■
■ ■■■■ task_4_check_if_trigger.py # task 4 ■ ■ ■■■■ task_base.py # classe base comum às
tasks ■ ■ ■■■■ output_database.py # conexões e engine do banco ■ ■■■■
download_imoveis_sicar_configuration/ ■ ■ ■■■■ dag_config.py # carrega configuração do
DAG ■ ■ ■■■■ dag-config.json # parâmetros (base_url, tabelas, conexão) ■ ■■■■
download_imoveis_sicar_utils/ ■ ■■■■ database_facade.py # abstração de acesso ao
Postgres ■ ■■■■ logger.py # logging das tasks ■ ■■■■ utils.py # utilitários (data, zip,
limpeza) ■■■■ venv/ # ambiente virtual do operador
```

3. Detalhamento por task e funções

Task 1 — 1_check_config · check_wfs_download_config.py

Garante que o ambiente esteja pronto antes de iniciar os downloads. Classe **CheackDownloadWFSConfig** (herda de TaskBase).

Função	Responsabilidade
verify_table_exists()	Verifica no schema public se todas as tabelas listadas em <code>required_tables</code> existem.
prepare_task()	Executa <code>dag_config()</code> e as verificações; lança exceção se algo estiver ausente.

Entrypoint: **check_wfs_download_config(project_dir)** — cria a configuração e retorna o resultado de `prepare_task()`.

Task 2 — 2_wfs_download · task_2_wfs_download.py

Baixa os shapefiles do SICAR via WFS. Classe **WFSDownload**. Para o ano corrente busca dois conjuntos por UF: registros **inseridos** no ano (`dat_criacao`) e registros **atualizados no dia** (`data_atualizacao`).

Função	Responsabilidade
is_first_cycle_run(logical_date)	Detecta se é a primeira execução do dia (via <code>DagRun.find</code>).
get_uf_list()	Se primeira execução, retorna todas as 27 UFs; senão, as UFs com <code>should_execute=true</code> .
get_period_by_uf(uf)	Define o ano a processar (<code>last_executed_year+1</code>), limitado ao ano corrente.
get_total_records(session, base_url, type_name, filters)	Consulta <code>numberMatched</code> do WFS para total de registros do filtro.
get()	Laço principal: para cada UF/filtro faz paginação (1000 features), baixa ZIPs e grava controle.
verify_file_exists(folder_path, file_name)	Checa se o arquivo já foi registrado em <code>sicar_shapefile_downloads</code> .
validate_and_cleanup_shapefile(...)	Compara contagem esperada vs. baixada; apaga registro/arquivo se divergir.
update_state_execution_control(...)	Atualiza <code>last_executed_year</code> e <code>shapefile_count/updated_shapefile_count</code> .
disable_uf_execution(uf)	Marca <code>should_execute=false</code> quando a UF já foi processada no ano corrente.
inset_download_record(...)	Insere registro do download (<code>state_code</code> , <code>year</code> , <code>directory_path</code> , <code>file_name</code>).
prepare_task(logical_date)	Orquestra o fluxo da task.

Entrypoint: **task_2_wfs_download(project_dir, logical_date)**.

Task 3 — 3_insert_shape_file · task_3_insert_shapefile.py

Descompacta, limpa e grava os shapefiles no Postgres. Classe **InsertShapeFile**. Registros com cod_imovel novo são inseridos em sicar_geometries; os que já existem vão para sicar_geometries_duplicates e depois atualizam os registros originais (UPSERT).

Função	Responsabilidade
get_shapefile_folder()	Seleciona diretórios pendentes (imported=false) em sicar_shapefile_downloads.
unzip_shapefiles(folders)	Descompacta os ZIPs (via Utils) dentro de cada pasta.
fix_axis_order(geom)	Inverte lon/lat -> lat/lon (transform do shapely).
insert_shapefile()	Lê cada .shp com geopandas, limpa dados (nulos, duplicados, Polygon->MultiPolygon), gera CSV temporário, carrega via COPY para tabela temporária e separa duplicados de novos.
update_imported_status(folder)	Marca imported=true após processar a pasta.
update_geometries()	UPDATE em sicar_geometries a partir de sicar_geometries_duplicates e limpa os duplicados.
prepare_task()	Orquestra: busca pastas, descompacta, insere, atualiza e apaga arquivos.

Entrypoint: **task_3_insert_shape_file(project_dir)**.

Task 4 — 4_check_if_trigger · task_4_check_if_trigger.py

Operador **ShortCircuitOperator** — se retornar False, interrompe o fluxo (trigger_self não executa). Classe **CheckIfTrigger**.

Função	Responsabilidade
check_if_trigger()	Conta UFs com should_execute=true em state_execution_control.
prepare_task()	Carrega configuração e retorna o resultado de check_if_trigger().

Entrypoint: **task_4_check_if_trigger(project_dir)**.

Módulos de apoio

Módulo / Classe	Responsabilidade
dag_config.py · DAG_Configuration	Carrega dag-config.json, monta conexão/engine, base_url, output_dir (Variable) e requerimentos.
task_base.py · TaskBase	Classe base: injeta DAG_Configuration e cria o logger (TasksLogger).
output_database.py · OutputDatabase	Recupera conexão do Airflow (BaseHook), cria DatabaseFacade e engine SQLAlchemy.
database_facade.py · DatabaseFacade	Abstração de banco: execute, fetchall, fetchone, tabelas, índices, etc.
logger.py · TasksLogger	Wrapper de logging com prefixo por task.
utils.py · Utils	get_today_date (timezone America/Sao_Paulo), unzip_shapefiles, delete_files.
dag-config.json	Parâmetros: logger_level, output_database, required_tables, base_url.

Tabelas envolvidas (PostgreSQL)

Tabela	Uso
sicar_shapefile_downloads	Controle dos downloads (state_code, year, directory_path, file_name, imported).
sicar_geometries	Tabela final de geometrias dos imóveis (cod_imovel é a chave de negócio).
sicar_geometries_duplicates	Registros cujo cod_imovel já existe — origem dos UPDATES.
state_execution_control	Controle por UF: last_executed_year, shapefile_count, updated_shapefile_count, should_execute.